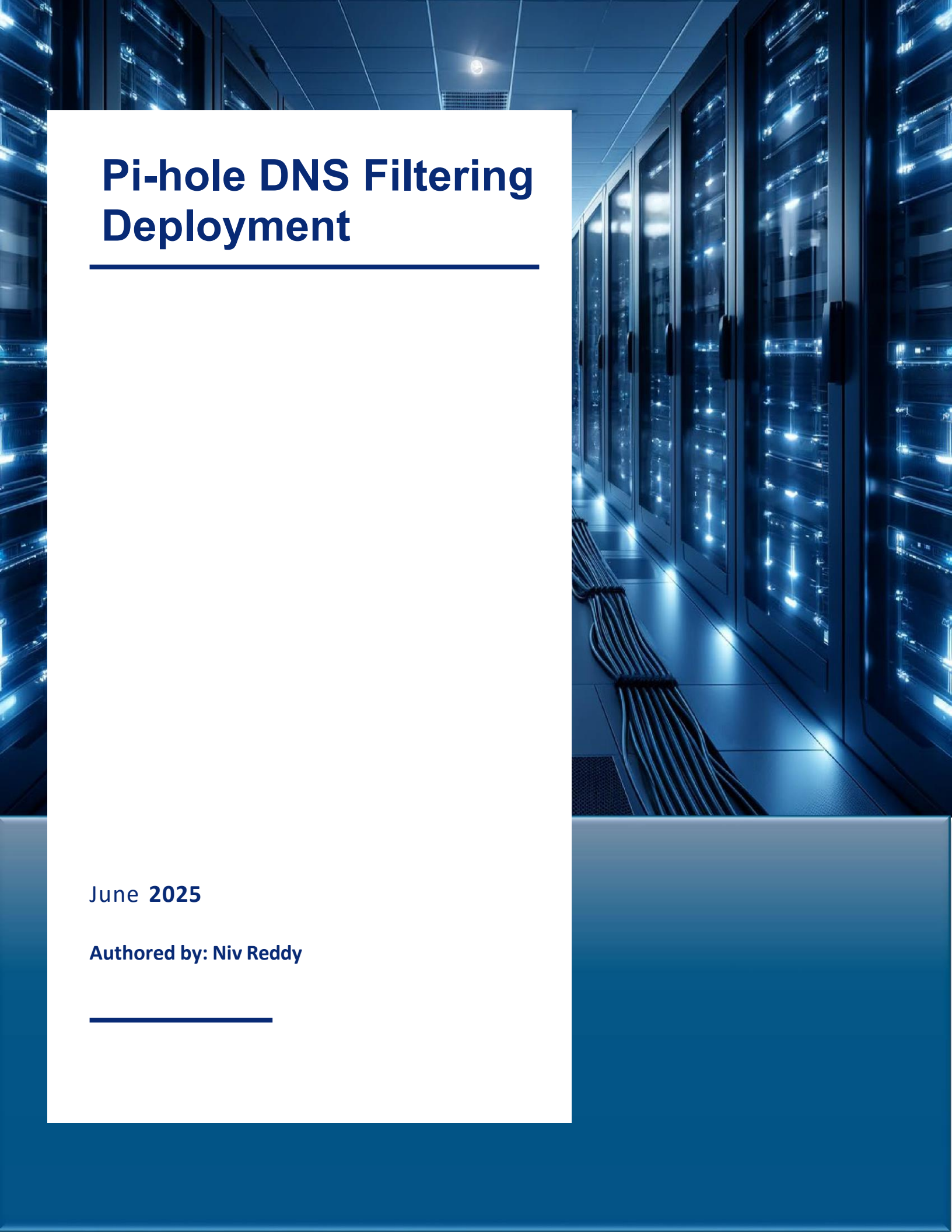




Pi-hole DNS Filtering Deployment

June 2025

Authored by: Niv Reddy

Introduction

Context

Previously, I was using the Eero Plus service for ad-blocking and content filtering. While convenient, it provided minimal transparency and customization over DNS queries or network filtering. I wanted more control, visibility, and security over DNS resolution across my home network.

Purpose

The goal of this project was to implement a more advanced and self-managed DNS filtering solution by deploying Pi-hole on a Raspberry Pi. This setup would allow for:

- Network-wide ad blocking
- DNS query logging
- Client-specific filtering policies
- Better understanding of how DNS impacts network security and performance

Equipment

Network Equipment and Software

Hardware Component	Description
Raspberry Pi 4 Model B (8GB RAM)	Main compute device running Pi-hole
Argon ONE M.2 Case	Aluminum case with passive cooling
SanDisk 64GB microSD	Storage medium for OS and Pi-hole
Smraza 5.1V 3A Power Supply	Reliable USB-C power source
Ethernet Cable	Ensures wired connectivity

Software Component	Description
Pi-hole v5.x	DNS-level ad blocker
Raspberry Pi OS Lite (64-bit)	Headless Raspberry Pi deployments
Web Admin Interface	Web dashboard managing and monitoring
SSH	Remote terminal access
Upstream DNS	Cloudflare 1.1.1.1

Configuration and Setup Process

Hardware Setup

- **Case Assembly:** Installed the Raspberry Pi 4 Model B into the Argon ONE M.2 case, ensuring proper alignment for passive cooling and cable routing.
- **Storage Preparation:** Flashed Raspberry Pi OS Lite (64-bit) to a SanDisk 64GB microSD card using Raspberry Pi Imager.
- **Configuration during Flashing:**
 - Set the hostname
 - Created a user account with a secure password
 - Disabled Wi-Fi to ensure wired-only operation
 - Enabled SSH using public key authentication (private key remains on personal Mac; public key was injected during flash)
- **Deployment:** Inserted the microSD card, assembled the case, and connected:
 - Power via USB-C (5.1V 3A)
 - Ethernet from the backhaul Eero node directly to the Pi's Ethernet port

Eero Network Configuration

- **IP Reservation:** Reserved a static IP for the Raspberry Pi in the Eero app under device settings
- **DNS Customization:**
 - Disabled Local DNS Caching in Eero
 - Changed custom DNS to point to the Pi-hole's reserved IP address instead of Cloudflare directly (i.e., all client queries now flow through Pi-hole first)

Configuration and Setup Process

Pi-hole Installation

- **Remote Access:** SSH'd into the Pi using the private key (from Mac) for passwordless and secure access
- **Software Installation:** Ran the official Pi-hole installation script:

```
curl -sSL https://install.pi-hole.net | bash
```

- **Setup Wizard:**
 - Assigned the static IP (matching the reserved IP in Eero)
 - Received the admin panel password on completion

Web Admin Dashboard Access

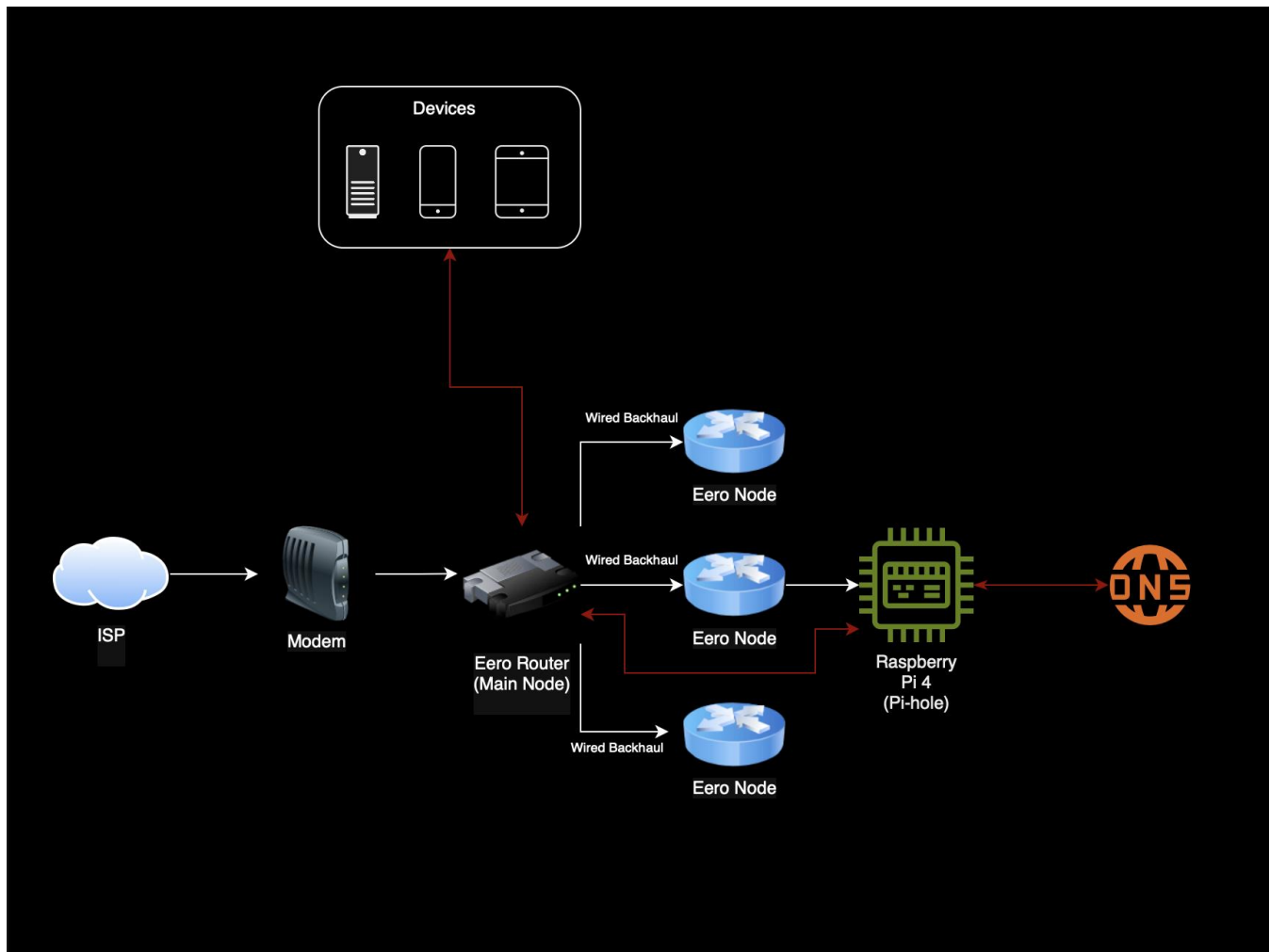
- Accessed via browser at

```
http://192.168.4.108/admin
```
- Logged in using the default admin password and immediately changed it for security
- Web UI now fully operational for monitoring, blocking, and DNS configuration

DNS Filtering and Functionality

Network Topology

Network Layout Diagram



Network Topology

Device Flow Explanation

The diagram above illustrates how DNS and network traffic flows within the home network. Here is a breakdown of each stage in the process:

1. **Devices (Clients):**

Laptops, phones, tablets, and other client devices generate DNS queries when accessing websites or services.

2. **Eero Mesh Network:**

These queries are forwarded to the main Eero router or the nearest Eero node via wired backhaul. The Eero system is configured to send all DNS requests to the Pi-hole instance instead of directly using an external DNS provider.

3. **Raspberry Pi 4 (Pi-hole):**

Pi-hole receives and processes all incoming DNS queries. It blocks domains based on configured blocklists (e.g., adware, trackers) and forwards allowed requests upstream to the DNS provider.

4. **Upstream DNS (Cloudflare 1.1.1.1):**

Pi-hole forwards clean queries to Cloudflare's DNS server (chosen for now). Cloudflare resolves the domain names into IP addresses and returns the responses to Pi-hole.

5. **Response Path:**

The Pi-hole sends the final DNS response back to the originating device through the Eero network, completing the resolution process.

This setup allows Pi-hole to act as a filtering layer for all DNS activity on the network, giving visibility, control, and performance benefits without disrupting device connectivity.

Monitoring and Maintenance